Gasballoon Cockpit

A single-file HTML web app for gas balloon flight planning and in-flight monitoring: live position tracking, wind-based landing predictions, staged descent planning, weather stations, air traffic, and offline map caching - built for use on a tablet in the basket.

Current version: v260812.03-1300 (12.08.2026) - this number always matches the `APP_VERSION` constant near the top of the script in `index.html`. Versioning scheme: `vYYMMDD.zz-HHMM` (date of the last change, a 2-digit counter that resets to 01 each new day, and the build time), so multiple same-day builds are unambiguous at a glance - the version is shown in the bottom-left corner of the app itself, useful for confirming a deployment actually picked up the latest build rather than a stale cached one. `cors_test.html`'s own version marker is kept in sync with this.

What it is

One HTML file, no build step, no server. Open it in a browser (or install it to a home screen as a PWA, with `apple-touch-icon.png`) and it runs entirely client-side, calling a handful of free public weather/mapping APIs directly from the browser. A service worker (`sw.js`) caches map *tile* requests only - deliberately not weather/elevation data, which needs to stay fresh - so previously-viewed map areas remain available if the connection drops.

Password gate on load (SHA-256 hashed, set in `GATE_HASH`).

Built for genuine 24/7 unattended operation: the service worker checks for its own updates every 30 minutes (not just on page reload, which might never happen once the app is left running), and a screen wake lock keeps the display on with automatic re-acquisition if the browser ever releases it.

The hamburger menu's last entry, "About - more info!", opens this README as a PDF in a small viewer page with always-visible Print/Close buttons (readme_viewer.html), rather than the browser's native PDF viewer directly.

Zoom-in/out buttons (and the very first automatic centring on GPS after boot) target the visually VISIBLE map area's centre, not the underlying map container's own geometric centre - relevant whenever the sidebar is open, since it overlays on top of the map rather than shrinking it.

Main functions

The app has two main functions, switched via a pill toggle at the top of the map.

Landing Area (default)

Shows the balloon's live predicted trajectory: a cruise segment (green) that transitions into a descent (yellow) at a configurable time ("Initiate descent in"), continuously recalculated as position, altitude, and wind data update. A Monte Carlo scatter of likely landing points is shown as a shaded polygon around the predicted landing point, with a black cross marking its actual centre (computed from the raw scatter itself, not the containing polygon's own vertices, which would skew it toward the edge). The map view gently follows this polygon as it shifts

in response to slider changes - only actually panning/zooming if it would otherwise fall outside the visible area, debounced so dragging a slider doesn't cause the map to jump on every intermediate value.

The descent point (a yellow cross on the trajectory) can be adjusted two ways that stay in sync: the "Initiate descent in" slider (10-minute steps, defaults to 20min), or dragging the cross itself along the trajectory.

Extended trajectory preview: click anywhere within roughly 60° of the trajectory's own direction, beyond where the normal prediction/landing area already reaches, to see the trajectory extended further out (blue) - a frozen snapshot from the moment of the click, useful for comparing the real flown track against an earlier prediction.

A one-time onboarding hint appears the first time this function is used.

Plan Descent

Two sub-functions, switched via a smaller toggle directly under the main "Plan Descent" button.

Quick Descent (default): click any point on the map to find the nearest reachable landing area and the descent path to it - shown as an orange cross (the calculated descent point) and a yellow descent path, same visual language as Landing Area's own live prediction. The reference trajectory truncates at the descent point once a plan is active. The descent point can be dragged to explore alternatives, or a new point clicked anywhere to restart. The reference trajectory itself extends as far as the "Initiate descent in" slider is set to.

Staged Descent: drag a royal-blue cross marker along the trajectory to choose when descent begins, then click within the resulting reachable area to search for a staged descent plan - one that pauses at one or more intermediate altitudes (chosen automatically where wind conditions genuinely differ) rather than descending continuously. Opens a side panel with a chart of the planned stages: altitude, wind at each level, duration, an estimated ballast requirement for each transition, and markers for any wind shear, inversion, or isothermal layers encountered. Minimum stage separation and maximum intermediate stages live directly in this panel. Its own two sliders (max descent time - 10-minute steps, defaults to 20min; inter-stage rate) sit side by side in one row.

Both sub-functions show a one-time onboarding hint on first use. Switching back to Landing Area clears the descent path from the map but leaves the reachable area, Monte-Carlo landing area, and its centre marker in place, with a "Clear staged descent area" button to remove them explicitly.

Flight data box

A small draggable panel showing course (always 3 digits, e.g. 065°), speed (km/h and kn), climb/sink rate, and altitude (m AMSL, ft, flight level above a configurable transition altitude, and AGL). Refreshes on a fixed 0.5s cadence, decoupled from the underlying GPS fix rate. Snaps flush against whichever map edge it's dragged past; defaults to the left edge just below the header button row. Dims along with the map in night mode.

Descent Parameters

"Initiate descent in" and "Descent rate" sit side by side. Descent rate's own track is a wedge shape that grows taller toward higher values, visualizing the accelerating descent speed - the draggable point itself still runs along a flat baseline; only the fill behind it

changes shape. Below each slider: the computed distance/effective rate, plus (for Descent rate specifically) the adiabatic lift bonus as a weight-equivalent ballast figure - the actual extra lift the adiabatic cooling effect provides during descent, typically a few kg. Intercept AGL, Rate below, and Scatter sit in a second row, each with its own computed time-to-reach badge.

Landing Area sidebar card

Shown for whichever landing-point marker is currently the relevant one (Landing Area's live prediction, Quick Descent's planned point, or Staged Descent's), with these fields:

- **Ground wind** at the predicted landing time, with wind gusts (when the model supports them) shown as a second line below it.
- **Terrain near landing point**: elevation-based roughness (flat / moderate slope / steep), nearby tagged buildings or forest, the specific land cover at the exact point (meadow, farmland, forest, residential, water, etc.), and a red "CAUTION WATER!" override if the point falls inside a mapped lake, river, or reservoir - checked via genuine polygon containment, not just proximity. This check runs on its own independent request queue, separate from the app's other Overpass-based features, so a failure elsewhere (airports, roads, place names) can't block it.
- **Sun at estimated landing**: azimuth (always 3 digits) and elevation, with a direction arrow that rotates to point at the sun's actual position, plus remaining time until evening civil twilight (HH:MM) - the more relevant cutoff than sunset itself for whether there's still enough light to see the terrain.
- **Moon phase**: a filled SVG silhouette (not an emoji, for consistent rendering across devices) with illumination percentage and a small waxing/waning trend arrow, plus rise/set times.

Weather modeling

Wind is fetched per-altitude across 19 pressure levels (18 for the two Météo-France models below, which don't publish 975hPa) from the auto-selected model for the current location (ICON-D2 for DACH, ICON-EU for wider Europe, GFS globally), refreshed on a timer (default 15min, configurable). CAPE and wind gusts are fetched as separate, independent requests on a reduced schedule (every 3rd wind refresh) so an unsupported variable on a given model can never take down the core wind fetch.

Besides the auto-selected models, ARPEGE Europe (Météo-France, 11km, all of Europe) and AROME France (Météo-France, 2.5km, France and immediate neighbours) can be picked manually for model comparison - both publish the pressure-level data this app needs. MeteoSwiss's own ICON-CH1/CH2 (1-2km, Switzerland) and the UK Met Office's models were deliberately not added: as of this writing neither publishes pressure-level/wind-at-altitude data through Open-Meteo, which the wind profile this app relies on requires.

Ensemble modeling: with a commercial Open-Meteo API key set (see below), the Monte-Carlo landing-area scatter in all three functions draws on real ensemble model members (ICON-D2-EPS / ICON-EU-EPS / GEFS, ~20-40 members depending on model) instead of an artificial age-based heuristic - each Monte-Carlo sample uses an actual member's own wind profile, genuine forecast uncertainty rather than a random perturbation. A small badge next to the model name in the header ("E" green / "H" gray) shows which mode is currently active. Falls back to the heuristic automatically without a key or if the ensemble fetch fails.

Power lines

Overhead line towers and transformers are rendered as small, muted markers rather than bright coloured circles, so they don’t visually compete with the lines themselves (able to be followed continuously) - spotting the wires is what actually matters for flight safety, not each individual mast along the route. Towers (neutral grey) and transformers (muted blue-grey) use distinct tones, not identical ones, so toggling one category’s checkbox in Settings while leaving the other on stays visually verifiable rather than looking like filtering stopped working. Category checkboxes and the “overhead only” toggle now force the layer to be fully removed and re-added rather than relying on VectorGrid’s own redraw() (documented upstream as unreliable for reapplying styles to already-rendered tiles) - this is very likely the actual reason unchecking a category didn’t always hide it.

Airspace

The ✈ airspace layer draws real, class-filtered airspace boundaries from openAIP’s Core API (confirmed working via a live CORS test - it was previously believed CORS-blocked) on top of the combined raster tile as a visual base (still the only way to show airports/navaids/reporting points, since openAIP retired separate per-category tile endpoints in 2023). The Settings checkboxes (Class A-G, Restricted, Prohibited, TMZ, RMZ) now genuinely filter which boundaries are drawn, refetched as the map moves or the selection changes. An airspace whose type/class isn’t recognized by the mapping used here is shown regardless of checkbox state, rather than silently hidden.

Commercial Open-Meteo API key: optional field in Settings. When set, every Open-Meteo request in the app (wind, CAPE, gusts, elevation, ensemble) automatically switches to the dedicated `customer-api.open-meteo.com` endpoint, which has no daily call limit, instead of the free tier’s 10,000/day cap - needed for genuine 24/7 unattended operation, since a single wind-profile request already counts as several calls toward that quota (any request covering more than 10 variables does). A running “Open-Meteo calls today” counter is shown in the weather model popover regardless of which tier is active.

Data sources

Source	Used for	Status
Open-Meteo	Wind profiles, elevation, CAPE, wind gusts, precipitation, ensemble members	Working. Optional commercial API key removes the free tier’s daily quota.
SondeHub	Radiosonde launches, for using real sounding data instead of the model	Working
api.existenz.ch (SwissMetNet)	Swiss weather stations	Working
Bright Sky	German weather stations	Working
MeteoGate/E-SOH	Weather stations across the rest of Europe (33 countries)	Station list loads; per-station value parsing not fully confirmed against a live response
ADSBExchange (via RapidAPI) + OGN/glidernet.org	Air traffic, deduplicated where both sources report the same aircraft	Working

RainViewer	Rain radar	Working
Overpass API	Airspace, terrain/land-cover, roads, region names, airports (multiple mirror fallbacks)	Working. The terrain check runs on its own independent queue; other features share a separate queue, so a failure in one can’t block the other.
Nominatim	Reverse geocoding for landing-area place names	Working
openAIP	Airspace tiles	Working
MetarCentral	Airport METARs	Working. Airport lookup uses a curated list of major European airports first, falling back to a live Overpass query (any OSM-tagged aerodrome with an ICAO code) elsewhere.
aprs.fi	APRS station tracking	**Not functional** - CORS-blocked from the browser, confirmed with a live key. Settings clearly label this.
Xweather	Lightning (only polls when rain is detected nearby)	Implemented but not yet confirmed against real credentials

Settings

Organized into color-coded groups: General (cache radius, transition altitude, units), Weather (model selection, sonde data, METAR/station sources, lightning), Air Traffic & Airspace, Terrain & Ground Features, Descent Planning (adiabatic model), Safety & Positioning (external GPS, emergency contact), and Data & Backup.

Tokens and keys: every external service credential the app uses, gathered in one place and individually editable - Open-Meteo (commercial), openAIP, RapidAPI (air traffic), aprs.fi, Xweather, EmailJS, Dropbox, GitHub. As with any client-side app, these are visible in the page source to anyone who views it; changing one asks for confirmation first.

Profile backup: export/import as a JSON file (native share sheet on mobile), or back up encrypted to a GitHub Gist (AES-GCM + PBKDF2, password-protected).

Emergency contact: pilot name, aircraft registration, mobile number, and email, with prepared-message sending over WhatsApp, SMS, or email (via mailto, or silently via EmailJS if configured).

Known limitations

- Adiabatic braking and Monte-Carlo scatter (without a commercial API key/real ensemble data) are approximations, not calibrated against real flight data - a planning aid, not a certified instrument.
- The landing-point terrain check is an approximation, not a true line-of-sight/obstacle-clearance calculation - ring-sampled elevation

plus OSM tags, since actual obstacle heights aren’t available from any free source used here.

- aprs.fi is confirmed non-functional (browser CORS restriction on their end, not fixable from this app).
- MeteoGate/E-SOH per-station value parsing and Xweather lightning are implemented but not yet confirmed against live credentials/responses.
- This is a static file with no server: after any change, it has to be re-uploaded to wherever it’s hosted before the live site reflects it. On iOS specifically, both the page itself and the service worker can be cached persistently enough that a hard reload or removing/re-adding the home-screen icon may be needed to pick up a new version.
- Cloud file pickers (Dropbox, Google Drive, etc.) shown when uploading a file are controlled by iOS/the browser based on installed provider apps, not by this page.